

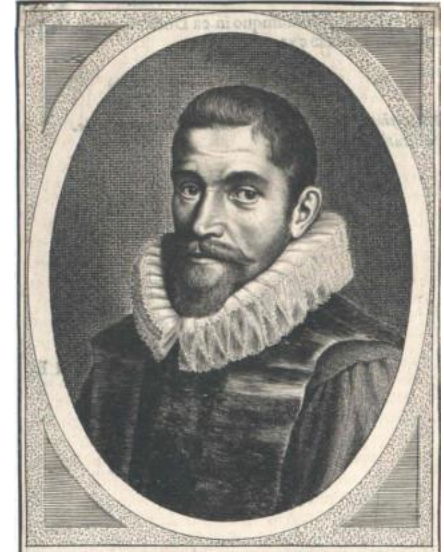
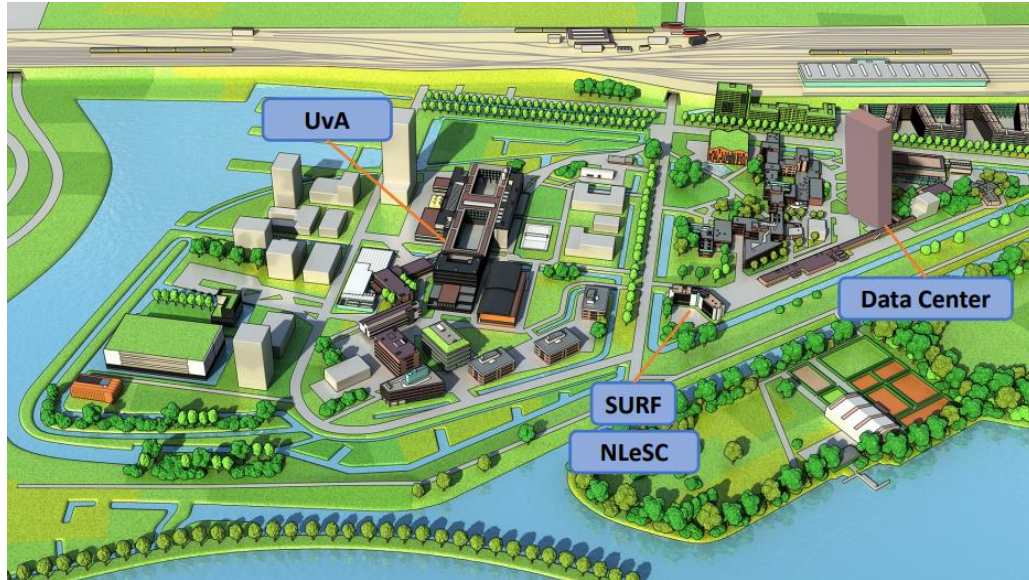
How to use Snellius



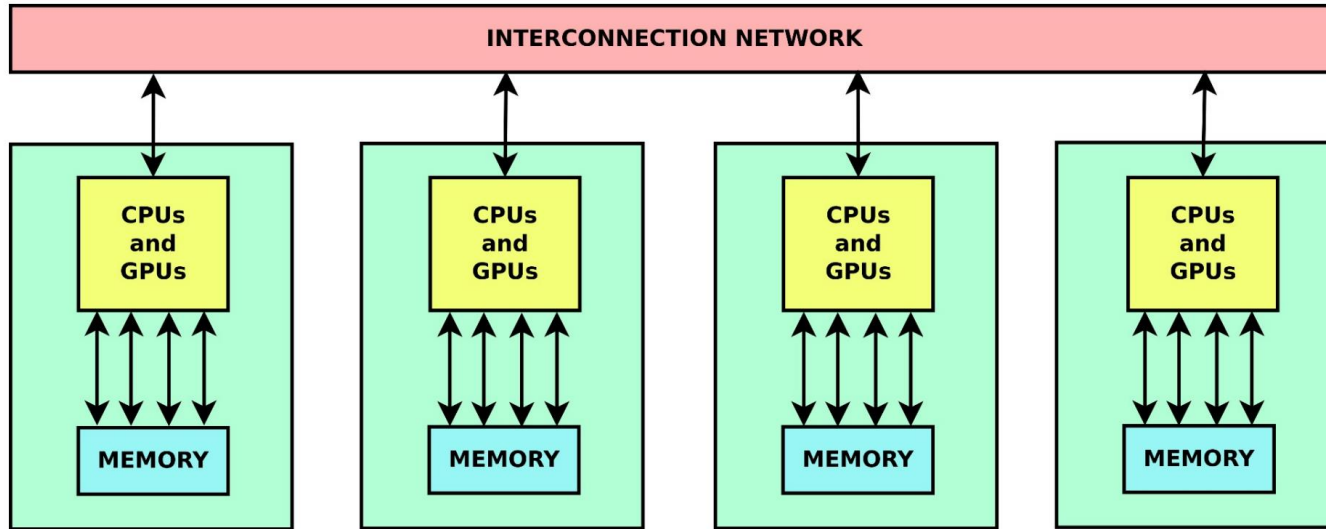
Introduction to Snellius

Snellius is the Dutch National supercomputer hosted at SURF.

A detailed description of the system can be found in the SURF [wiki](#).



What is a supercomputer?



Credits: SURF team

Supercomputer/High Performance Computer (HPC) = a cluster of big computers (servers/nodes)

Using a supercomputer

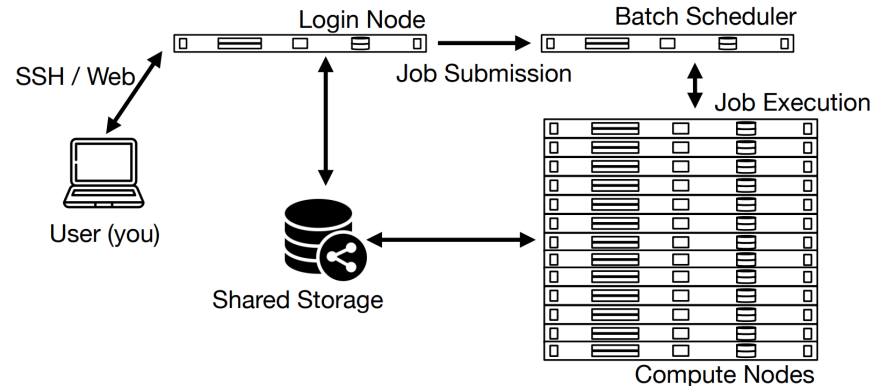
- Interacting with a cluster computer is **different** from a normal computer. Normal computers are mostly used interactively, i.e. you type a command or click with your mouse, and your computer instantly responds by e.g. running a program. Cluster computers are mostly used non-interactively.



User (you)

Using a supercomputer

- A cluster computer such as Snellius mainly has two types of nodes: **login nodes** and **compute nodes**.
 - **login nodes:** Entrance point. There are only a few login nodes on a cluster computer, and you only use them for light tasks, many users can be on the same login node at the same time.
 - **Compute nodes:** From the login node your job script is submitted to the job queue. As soon as sufficient resources (i.e. compute nodes) are available for your job, the system will take your job from the queue and send it to the compute nodes for execution.



Access to Snellius login nodes

Connect to Snellius login nodes:

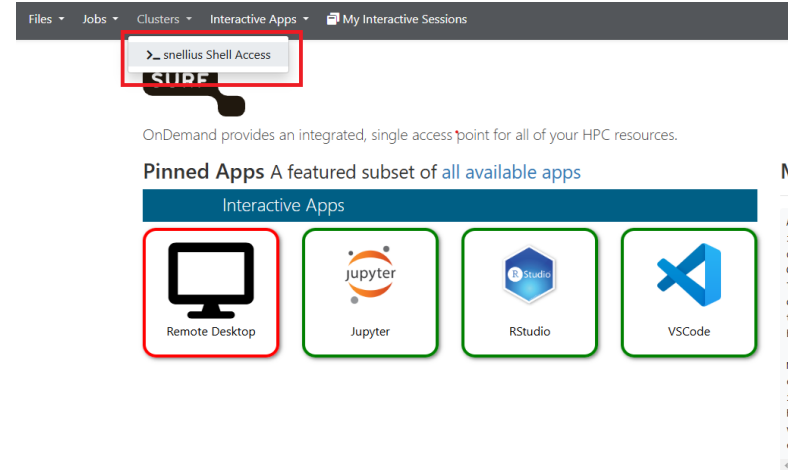
- You can use either the web version from **OnDemand**
- Or connect with terminal secure shell (SSH):
ssh <username>@snellius.surf.nl

After successful login

```
nct00021@login1:~>
```

↑
Login name

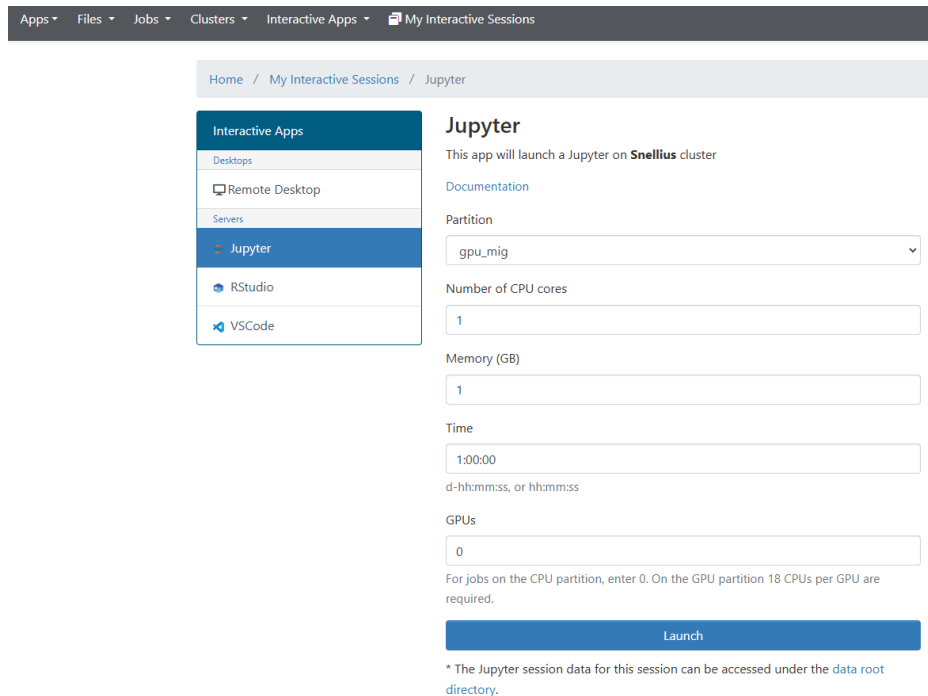
↙
Host name



Access to Snellius compute nodes

Connect to Snellius compute nodes:

- **Schedule jobs** (recommended, not burning your credits when not computing)
- **Interactive Apps** (keep burning your credits even when not computing)
 - Jupyter
 - VS Code
 - Interactive access to compute node



The screenshot shows the Snellius web interface for launching interactive applications. The top navigation bar includes 'Apps', 'Files', 'Jobs', 'Clusters', 'Interactive Apps', and 'My Interactive Sessions'. The breadcrumb trail is 'Home / My Interactive Sessions / Jupyter'. On the left, a sidebar titled 'Interactive Apps' lists 'Desktops', 'Remote Desktop', 'Servers', 'Jupyter' (selected), 'RStudio', and 'VSCode'. The main content area is titled 'Jupyter' and states 'This app will launch a Jupyter on Snellius cluster'. It includes a 'Documentation' link and configuration fields: 'Partition' (set to 'gpu_mig'), 'Number of CPU cores' (set to '1'), 'Memory (GB)' (set to '1'), 'Time' (set to '1:00:00'), and 'GPUs' (set to '0'). A note specifies that for CPU partitions, 0 should be entered, while GPU partitions require 18 CPUs per GPU. A blue 'Launch' button is at the bottom, followed by a footnote: '* The Jupyter session data for this session can be accessed under the data root directory.'

Login node

Check your remaining budgets: *accinfo*

```
[scur2916@int5 ~]$ accinfo
# Account information

Account          : jhssrf015
Account expiry    : 2025-06-30
Contact first name : Caspar
Contact last name  : van Leeuwen
Contact email     : caspar.vanleeuwen@surf.nl

# Active budgets linked to this account

Budget name       : Internal-Snellius-Jupyter-2023/L1/37
Budget expiry     : 2024-12-31
Project           : Internal-Snellius-Jupyter-2023
Excess usage allowed : no
Initial budget    : 10000:00 SBU
Used budget       : 238:34 SBU
Remaining budget   : 9761:25 SBU
Products          : cbuidl, gpu_course, staging, staging_course, thin_course

Budget name       : internal-test-dev-2023/L1/11
Budget expiry     : 2025-06-30
Project           : internal-test-dev-2023
Excess usage allowed : no
Initial budget    : 10000:00 SBU
Used budget       : 1462:05 SBU
Remaining budget   : 8537:54 SBU
Products          : gpu, gpu_a100, gpu_h100, gpu_mig

# Budgets linked to this account
```

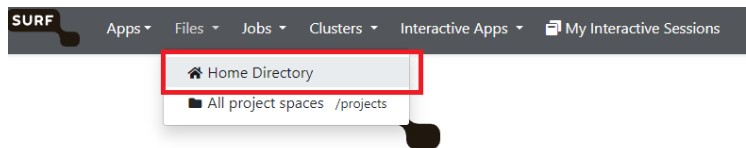

Login node - Exercise

- Explore the file system: *pwd*, *ls*, *cd*
- Create and edit files/directories: *mkdir*, *cp*, *mv*, *nano*, *rm*
- Use env variables: *\$USER*, *\$HOME*

```
[scur2916@int5 ~]$ pwd
/home/scur2916
[scur2916@int5 ~]$ echo $USER
scur2916
[scur2916@int5 ~]$ echo $HOME
/home/scur2916
[scur2916@int5 ~]$ mkdir example/
[scur2916@int5 ~]$ cd example/
[scur2916@int5 example]$ ls
[scur2916@int5 example]$ cd ..
[scur2916@int5 ~]$ rm -r example/
[scur2916@int5 ~]$ cd example/
-bash: cd: example/: No such file or directory
[scur2916@int5 ~]$ nano aboutme.txt
```

Data transfer

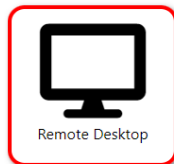
Transfer your data with ondemand Home Directory/git pull request/[using the local terminal scp](#)



OnDemand provides an integrated, single access point

Pinned Apps A featured subset of **all** ava

Interactive Apps



Remote Desktop



Jupyter

To copy a file *sourcefile* to the `$HOME/destinationdir` folder on the HPC systems, use

Snellius

```
scp sourcefile <username>@snellius.surf.nl:destinationdir
```

Alternatively, you can rename the file on the HPC system (it will overwrite the existing *destinationfile* if it exists) by issuing the command

Snellius

```
scp sourcefile <username>@snellius.surf.nl:destinationdir/destinationfile
```

To copy a complete directory, use the `-r` argument, e.g.

Snellius

```
scp -r sourcedir <username>@snellius.surf.nl:destinationdir
```

This will create the directory `destinationdir/sourcedir` on the HPC system and copy the contents of your local *sourcedir* to that new directory.

To copy from the HPC system to your local PC, simply reverse the order of arguments. For example, to copy the file *sourcefile* from the remote home directory to the local home directory (`~`), use

Snellius

```
scp -r <username>@snellius.surf.nl:sourcefile ~
```

Data transfer - Exercise

- Git clone the following repo:
 - **git clone https://gitlab.tue.nl/hpclab/courses/snellius-introduction.git**
- Explore the git repository
 - **cd snellius-introduction/**
 - **ls**

```
settek@TUE035721:~$ git clone https://gitlab.tue.nl/hpclab/courses/snellius-introduction.git
Cloning into 'snellius-introduction'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
settek@TUE035721:~$ cd snellius-introduction/
```

Use of library

Always check if the libraries you need is already available on HPC, because:

- ☐ Optimized for the hardware of the supercomputer
- ☐ Software is available as modules
- ☐ A module automatically loads dependencies
- ☐ Reduce **waste** of resources for the same library

```
[scur2916@int5 ~]$ module avail  
  
----- /sw/noarch/environment -----  
2020_SABS 2022 2023 2024 EESSI/2023.06  
  
If the avail list is too long consider trying:  
  
"module --default avail" or "ml -d av" to just list the default modules.  
"module overview" or "ml ov" to display the number of modules for each name.  
  
Use "module spider" to find all possible modules and extensions.  
Use "module keyword key1 key2 ..." to search for all possible modules matching any of the "keys".
```

Use of library - exercise

- Useful commands:
- *module avail* - available modules in the system
- *module load <mod>* - load <mod> in the shell environment
- *module list* - show a list of all loaded modules
- *module unload <mod>* - remove <mod> from the environment
- *module purge* - unload all modules
- *module whatis <mod>* - show information about <mod>

```
[scur2916@int5 ~]$ module load 2023
[scur2916@int5 ~]$ module avail PyTorch

-----
PyTorch/2.1.2-foss-2023a-CUDA-12.1.1  PyTorch/2.1.2-foss-2023a

If the avail list is too long consider trying:

"module --default avail" or "ml -d av" to just list the default modules.
"module overview" or "ml ov" to display the number of modules for each name.

Use "module spider" to find all possible modules and extensions.
Use "module keyword key1 key2 ..." to search for all possible modules matching any of the

[scur2916@int5 ~]$ module load PyTorch/2.1.2-foss-2023a-CUDA-12.1.1
[scur2916@int5 ~]$ module list

Currently Loaded Modules:
  1) 2023
  2) GCCcore/12.3.0
  3) zlib/1.2.13-GCCcore-12.3.0
  4) binutils/2.40-GCCcore-12.3.0
  5) GCC/12.3.0
  6) numactl/2.0.16-GCCcore-12.3.0
  7) XZ/5.4.2-GCCcore-12.3.0
  8) libxml2/2.11.4-GCCcore-12.3.0
  9) libpciaccess/0.17-GCCcore-12.3.0
 10) hwloc/2.9.1-GCCcore-12.3.0
 11) OpenSSL/3
 12) libevent/2.1.12-GCCcore-12.3.0
 13) UCX/1.14.1-GCCcore-12.3.0
 14) libfabric/1.18.0-GCCcore-12.3.0
 15) UCC/1.2.0-GCCcore-12.3.0
 16) OpenMPI/4.1.5-GCC-12.3.0
 17) OpenBLAS/0.3.23-GCC-12.3.0
 18) FlexiBLAS/3.3.1-GCC-12.3.0
 19) FFTW/3.3.10-GCC-12.3.0
 20) gomp/2023a
 21) FFTW.MPI/3.3.10-gomp-2023a
 22) ScaLAPACK/2.2.0-gomp-2023a-fb
 23) foss/2023a
 24) CUDA/12.1.1
 25) cuDNN/8.9.2.26-CUDA-12.1.1
 26) GDRCopy/2.3.1-GCCcore-12.3.0
 27) UCX-CUDA/1.14.1-GCCcore-12.3.0-CUDA-12.1.1
 28) magma/2.7.2-foss-2023a-CUDA-12.1.1
 29) NCCL/2.18.3-GCCcore-12.3.0-CUDA-12.1.1
 30) Ninja/1.11.1-GCCcore-12.3.0
 31) bzip2/1.0.8-GCCcore-12.3.0
 32) ncurses/6.4-GCCcore-12.3.0
 33) libreadline/8.2-GCCcore-12.3.0
 34) Tcl/8.6.13-GCCcore-12.3.0
```

Use of library – what if the library is not available?

- If the package you need is not included in existing modules, first create a virtual environment before pip install

1) *python -m venv my_venv*

2) *source my_venv/bin/activate*

3) *pip install...*

```
settek@TUE035721:~/snellius-introduction$ python -m venv my_venv_  
settek@TUE035721:~/snellius-introduction$ source my_venv\Scripts\activate
```

Submit your jobs on Snellius

- Create a batch job script, for an example look **example.sh** (cat example.sh) in **snellius-introduction.git**

```
#!/bin/bash
#SBATCH --partition=gpu_mig
#SBATCH --gpus=1
#SBATCH --job-name=CheckEnvironment
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=9
#SBATCH --time=00:05:00
#SBATCH --output=slurm_output_%A.txt
#SBATCH --error=slurm_error_%A.txt

# Load necessary modules (adjust based on your environment)
module purge
module load 2023
module load PyTorch/2.1.2-foss-2023a-CUDA-12.1.1

source my_venv/bin/activate # activate your virtual environment
# pip install --no-cache-dir -r requirements.txt

srun python -uc "import torch; print('GPU available?', torch.cuda.is_available())"
```

Submit your jobs on Snellius - exercise

Useful commands	Explained
<code>sbatch <jobscrip></code>	submit a job to the scheduler
<code>squeue -j <job_id></code>	inspect the status of job <job_id>
<code>squeue -u <user_id></code>	inspect all jobs of user <user_id>
<code>scancel <job_id></code>	cancel job <job_id> before it runs
<code>scontrol show job <job_id></code>	show estimated job start

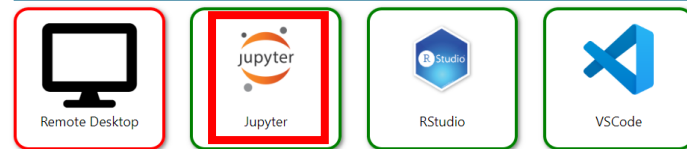
```
[scur2916@int5 ~]$ sbatch example.sh
```


Submit your jobs on Snellius – job arguments

Job arguments	Explained
<code>#SBATCH --partition=gpu_mig</code>	<i>The partition of Snellius on which you want to run your job.</i>
<code>#SBATCH --gpus=1</code>	<i>Number of GPUs you request from the nodes. If you haven't designed your code to explicitly run on multiple GPUs, please use only one GPU</i>
<code>#SBATCH --job-name=CheckEnvironment</code>	<i>Name of the job to pop up when you list your jobs with <code>squeue</code></i>
<code>#SBATCH --ntasks=1</code>	<i>Number of tasks to run with the job.</i>
<code>#SBATCH --cpus-per-task=9</code>	<i>Number of CPUs you request from the nodes. The <code>gpu_mig</code> partition restricts you to max. 9 CPUs per job/GPU.</i>
<code>#SBATCH --time=00:05:00</code>	<i>Estimated time your job needs to finish. It is no problem if your job finishes earlier than the specified time. However, if your job takes longer, it will be instantaneously killed after the specified time. Still, don't specify unnecessarily long times as this causes your job to be scheduled later (you need to wait longer in the queue if other people also want to use the cluster). A good rule of thumb is to specify ~20% more than what you would expect.</i>
<code>#SBATCH --output=slurm_output_%A.txt</code>	<i>Output file to which the slurm output should be written. The tag “%A” is automatically replaced by the job ID.</i>
<code>#SBATCH --error=slurm_error_%A.txt</code>	<i>Error file</i>

Pinned Apps A featured subset of [all available apps](#)

Interactive Apps



Jupyter

This app will launch a Jupyter on **Snellius** cluster

[Documentation](#)

Partition

gpu_mig

Number of CPU cores

1

Memory (GB)

1

Time

1:00:00

d-hh:mm:ss, or hh:mm:ss

GPUs

1

For jobs on the CPU partition, enter 0. On the GPU partition 18 CPUs per GPU are required.

Reservation

jhs_tue2022

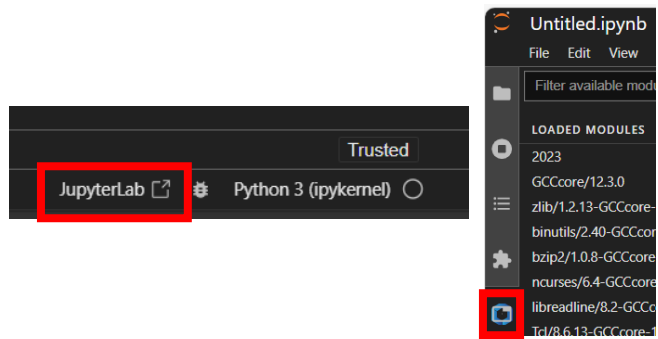
Launch

Use Jupyter notebooks

- Go to [Ondemand Snellius](#) and login with your Snellius credentials
- Go to **Jupyter** tile
- Launch from GPU partition
 - CPU number and memory do not matter
 - GPU number does matter
 - You can use the **reserved nodes** for free (only gpu_mig) from reservation
- Start or create **notebook**

TIPS:

- You can (un)load **modules** by going to Jupyterlab -> Softwares (left menu bar)



Final exercise

- Git clone <https://gitlab.tue.nl/hpclab/courses/snellius-introduction.git>
- Have a look at the python script and the slurm job
 - **cat gpu_pytorch_mnist.py**
 - **cat exercise.sh**
- Schedule job on reserved node
 - **sbatch exercise.sh**
- Follow the status of the job
 - **squeue**
- When the job is finished, check out the output file
 - **cat %x_%j.out**

Further learning: speed up your training with scratch node

- Expert advice: Run your job on `$TMPDIR` - When you store your datasets in your home directory, the read/write operation might become a bottleneck of your job. How to do it? [link](#)

File system	Quota (space)	Quota (files)	Speed	Shared between nodes	Mount point	Expiration	Backup	Notes
Home	200 GiB	1,000,000	Moderate, Normal	Yes	/home/<username>	15 weeks after project expiration	Nightly incremental	
Scratch-local	8 TiB (counted over all scratch-* space used)	3,000,000 (soft limit)	Fast	No	/scratch-local/<username>	Files older than 6 days are removed automatically	No backup	
Scratch-shared	8 TiB (counted over all scratch-* space used)	3,000,000 (soft limit)	Fast	Yes	/scratch-shared/<username>	Files older than 14 days are removed automatically	No backup	
Scratch-node	none	none	Very fast	No	/scratch-node/<user-specific>	When the job ends the file system is deleted . You cannot retrieve any data from this file system after your job ends.	No backup	All fcn nodes, a subset of tcn nodes, and a subset of gcn nodes have this file system. These nodes can be requested with <code>#SBATCH --constraint=scratch-node</code> Environment variable <code>TMPDIR</code> will point to a user-specific temporary directory in <code>/scratch-node</code> in this case. See below for more details.

Further learning: advices

- Some models and datasets are already available on Snellius: [link](#)
- Anaconda is not recommended on snellius: [Beware: the anaconda is squeezing us](#)
- TU/e Supercomputing Center organizes a Spring and Autumn HPC School: [HPC School Events](#)
- All my jobs are pending: [link](#)
- ***More information about snellius please refer to [SURF Wiki](#)***

Feel free to reach out



<https://supercomputing.tue.nl/>

Email: hpcsupport@tue.nl (we will help you!)